

GeoGebra 几何脚本工作台综合科技编程项目实践报告

摘要

- 研究以及项目创建维护者：吴茂源 济宁市兖州区实验高级中学附属学校共产主义青年团团员
- 文章撰写于2026.5.2日
- 我本人坚定理想信念，铸牢中华民族共同体意识，坚定对伟大祖国、中华民族、中华文化、中国共产党、中国特色社会主义的高度认可，积极参加少先队、共青团活动，学习和践行社会主义核心价值观
- 开源地址：<https://github.com/Chloemlla/GeoGebra-Script-Lab>

本项目是一个基于 GeoGebra Web API 的交互式几何脚本工作台。用户可以在左侧编写几何指令，在中间查看画布渲染效果，在下方查看执行日志，并通过拖拽自由点把新的坐标同步回代码。项目同时接入了账号系统、项目空间、版本快照、AI 生成、导出任务、团队协作、审阅评论和管理员后台，形成了从“输入脚本”到“生成图形”再到“分享和管理”的完整闭环。

本报告依据当前仓库源码整理，重点说明系统目标、技术路线、模块设计、关键算法、遇到的问题以及解决方案。整体上，前端采用 React 19 + Vite + Monaco Editor + GeoGebra Web API，后端采用 Rust + Hyper + Tokio + MongoDB + Reqwest，并通过统一的 JSON Envelope、任务队列、缓存与权限控制把功能组织成一个可扩展的综合科技编程项目。

关键词

GeoGebra, React, Monaco Editor, Rust, Hyper, Tokio, MongoDB, AI 生成, 几何建模, 综合科技编程

1. 项目背景与研究意义

初中阶段的数学学习常常需要“看见”几何关系，但传统纸笔绘图存在操作慢、修改难、无法联动公式和对象关系的问题。本项目把几何作图过程转化为可执行脚本，让学生能够像写程序一样描述图形结构，再由 GeoGebra 画布自动渲染结果。

这个设计有三个意义：

- 把“几何”变成“可编程对象”，让数学学习和编程思维结合。
- 把“画图”变成“脚本执行”，使图形、日志、版本和分享都可追踪。
- 把“演示工具”升级为“学习平台”，支持讲解模式、演示模式、AI 辅助解释和团队协作。

2. 项目目标

项目最初的目标不是单纯做一个画板，而是完成一个完整的几何脚本工作台。核心目标包括：

- 允许用户输入 GeoGebra 指令并实时运行。
- 通过预处理器清洗、校验和统计脚本。
- 通过执行器按顺序执行命令并记录日志。
- 支持自由点拖拽后回写到代码，形成“代码-图形”双向闭环。
- 支持登录、注册、项目保存、版本快照和云端同步。

6. 支持 AI 生成图形脚本、图形解读、对象解释和教学标注。
7. 支持导出 SVG、PDF、GIF、MP4 等结果，并保留可分享链接。
8. 提供管理员后台，观察任务队列、缓存、模型状态和 MongoDB 状态。

3. 总体架构

项目采用分层架构，清晰划分了“表现层、逻辑层、服务层和存储层”。



这一架构的优点是边界清晰：

1. 前端负责交互和展示。
2. 逻辑层负责脚本清洗、调度与状态判断。
3. 后端负责认证、持久化、AI 调用、任务队列和管理能力。
4. 存储层负责本地缓存、服务端数据库和临时导出文件。

4. 前端实现

4.1 入口与页面路由

前端入口是 `src/index.jsx`，它把 App 挂载到 `#root`。页面路由由 `src/utils/appRoutes.js` 和 `src/hooks/useAppRoute.js` 共同完成，支持四个主要页面：

1. `auth`：登录与注册页。
2. `overview`：项目概览页。
3. `studio`：脚本工作台页。
4. `backend`：后端能力与管理页。

路由设计是单页应用式的，但没有依赖复杂路由库，而是通过 `history.pushState`、`popstate` 和 `share` 查询参数完成切换。这样做简单、可控，也更适合学生理解前端状态与地址栏的关系。

4.2 Monaco 编辑器

编辑器封装在 `src/components/MonacoCodeEditor.jsx` 和 `CodeEditor.jsx` 中，使用 Monaco Editor 作为代码编辑核心，具备：

1. 语法高亮。
2. 行号。
3. 折叠。
4. 自动换行。
5. 自定义主题 `geogebra-workbench`。
6. 自定义语言 `geogebra`。

编辑器通过 Web Worker 提升稳定性，且只在需要时按需加载，避免首屏太重。主题和词法规则专门针对 GeoGebra 指令做了优化，例如识别 `Polygon`、`Circle`、`Segment`、`Midpoint`、`Text`、`Translate`、`Rotate`、`Tangent` 等关键字。

4.3 GeoGebra 容器

`src/components/GeoGebraContainer.jsx` 负责加载 GeoGebra 画布，关键点是异步初始化：

1. 先监听容器宽度。
2. 再等待 `deployggb.js` 和 `window.GGBApplet` 就绪。
3. 通过 `ggbOnInit` 回调确认 Applet 加载完成。
4. 初始化成功后再触发 `onReady`。

为了提升体验，容器还做了骨架屏、进度条、错误提示和自动重试提示。这样即使 GeoGebra 加载较慢，用户也不会觉得页面“卡死”。

4.4 控制面板与日志面板

`ControlPanel` 提供运行、清空、导出、重置和画布模式切换。`LogPanel` 负责按时间记录每条指令的执行结果，并自动滚动到最新位置。

控制区的设计思路很直接：

1. 所有高风险操作都要可见。
2. 运行前自动重置画布。
3. 运行状态、成功数、错误数和耗时都要显示。
4. 失败时必须有明确日志，而不是只弹一个“失败”。

4.5 工作台功能

`App.jsx` 把页面组织成多个工作区块，包含：

1. 基础脚本示例。
2. 项目空间与版本快照。
3. 参数面板。
4. 批量样式。
5. 讲解模式和演示模式。
6. 拖拽回写增强。

7. AI 图形解读和对象解释。

8. 导出矩阵和分享发布。

这说明项目已经不是一个简单的“画图工具”，而是一个完整的几何学习与创作平台。

5. 核心技术实现

5.1 代码预处理器

`src/engine/Preprocessor.js` 的作用是把用户输入的脚本清洗成可以执行的命令序列。它做了四类工作：

1. 去掉空行。
2. 去掉注释。
3. 检查括号匹配和赋值格式。
4. 做安全限制与风险提示。

其中最关键的一点是：它不会把字符串里的 `//` 当成注释。例如下面这种情况要保留原文：

```
A = Text("http://example.com", (0, 0))
```

处理思路是先找到 `//`，再统计前面引号数量，判断它是否处于字符串内部。这样可以减少误删。

预处理器还设置了上限：

1. 最大代码长度 `50000` 字符。
2. 最大命令数 `5000` 行。
3. 最大嵌套深度 `3`。
4. 对 `Sequence`、`Iteration`、`RecursiveSequence` 这类高风险函数发出警告。

这相当于给脚本加了“防护栏”，防止浏览器因超长或超复杂脚本而卡死。

5.2 指令调度器

`src/engine/Dispatcher.js` 负责顺序执行命令，并记录执行轨迹。它的工作流程是：

1. 检查引擎是否已就绪。
2. 可选地先重置画布。
3. 逐行执行命令。
4. 每行都记录成功、失败、警告和耗时。
5. 如果出现错误，停止后续执行，避免级联错误。

调度器还设置了总执行超时 `30000ms`，这样即使脚本不合法，也不会把浏览器拖死。

5.3 GeoGebra 引擎封装

`src/engine/GeoGebraEngine.js` 是项目最核心的封装层，负责和 GeoGebra Web API 对接。它解决了几个关键问题：

1. 异步加载问题：通过 `readyPromise`、`ggbOnInit` 和超时控制处理。
2. 语言问题：强制使用英文 `language: 'en'`，避免本地化命令识别失败。
3. 命令执行可靠性：不只看 `evalCommand` 返回值，还会比较执行前后对象数量和目标对象是否出现。

4. 自由点同步：识别 `movedGeos` 事件，把拖拽结果回写到脚本。
5. 样式控制：支持颜色、线宽、点大小、标签显示、网格、坐标轴等批量设置。

特别值得注意的是，执行成功判断不是单一布尔值，而是多重确认：

1. `evalCommand(command)` 的结果。
2. 执行前后对象集是否变化。
3. `evalCommandGetLabels` 是否返回了新标签。

这样可以减少 GeoGebra API 在复杂场景下“返回值不够直观”的问题。

5.4 拖拽回写与画布漂移

项目支持用户在画布上拖拽自由点，再把新坐标同步回代码。相关逻辑包含：

1. `isDraggableFreePoint` 判断对象是否是独立且可拖拽的点。
2. `exportFreePointsAsCode` 导出点的坐标表达式。
3. `buildPointCommandDiffs` 比较原代码与画布当前状态。
4. `mergeCanvasStateIntoCode` 把拖拽后的点位合并回脚本。

这套机制解决了“图形变了，但代码没变”的问题，也避免学生每次拖拽后都要手动抄坐标。

5.5 参数面板、样式面板和讲解模式

`src/utils/studio.js` 提供了很多实用工具：

1. 数值变量可以自动变成滑块和数字输入框。
2. 布尔变量可以自动变成开关。
3. 字符串变量可以自动变成文本输入框。
4. 批量样式可以统一修改颜色、线宽、点大小和标签可见性。
5. 讲解模式可以按顺序逐步构造图形，适合课堂演示。
6. 演示模式可以隐藏代码区，只保留画布。

这些功能让这个项目从“程序”变成了“课堂工具”。

6. 后端实现

6.1 Rust 服务入口与配置

后端入口是 `backend/src/main.rs` 和 `backend/src/app.rs`。服务使用 `Tokio` 异步运行，默认监听 `127.0.0.1:3001`，也可以通过环境变量修改：

1. `BIND_ADDR`
2. `API_BASE_URL`
3. `MODEL_BASE_URL`
4. `MODEL_NAME`
5. `API_KEY`
6. `MONGODB_URI`
7. `MONGODB_DATABASE`
8. `FRONTEND_DIST_DIR`

模型服务默认对接 OpenAI 兼容接口，默认模型名为 `gpt-4.1-mini`，但可通过环境变量替换。

6.2 统一响应格式

后端所有接口都使用统一的 Envelope 结构：

```
{
  "success": true,
  "code": "OK",
  "message": "service is healthy",
  "requestId": "req_xxx",
  "data": {},
  "meta": {
    "timestamp": "2026-04-25T14:10:00Z",
    "version": "v1"
  },
  "error": null
}
```

这种做法的好处是前端处理简单，错误排查也更统一。

6.3 认证系统

认证逻辑位于 `backend/src/auth.rs`。它包含：

1. 注册。
2. 登录。
3. 会话查询。
4. 退出登录。

安全细节包括：

1. 邮箱、用户名和密码格式校验。
2. 密码使用 Argon2 哈希。
3. 会话 token 有有效期，默认 30 天。
4. 第一个注册用户自动成为管理员，便于系统初始化。

前端在 `src/api/backend.js` 中保存 token，并在请求头里自动加入 `Authorization: Bearer ...`。

6.4 存储策略

后端采用“内存优先，MongoDB 可选”的设计：

1. `MemoryStore` 用于本地运行和快速开发。
2. `MongoStore` 用于持久化和多用户环境。
3. 如果 MongoDB 没有配置，系统仍能工作，只是数据不会跨重启保存。

MongoDB 中建立了多个索引，保证查询和唯一性：

1. `shares.slug` 唯一索引。
2. `teams.slug` 唯一索引。
3. `users.email` 唯一索引。
4. `users.username` 唯一索引。

5. `sessions.token` 唯一索引。

6. `sessions.expiresAt` TTL 索引。

这说明项目不仅能跑，还考虑了真实部署中的性能和数据一致性。

6.5 AI 生成与对象解释

`backend/src/model.rs` 负责模型调用，走的是标准的 `chat/completions` 风格接口，并要求模型直接返回 JSON。项目支持三种 AI 能力：

1. `drawing-jobs`：根据图片或提示词生成 GeoGebra 脚本。
2. `script-insights`：解释整段脚本。
3. `object-explanations` / `annotation-jobs`：解释对象依赖与生成标注建议。

如果模型返回失败或内容不完整，系统会退回本地 fallback 方案，保证流程不断线。

6.6 导出任务

导出逻辑分为两层：

1. 结构化导出：SVG、PDF。
2. 媒体导出：GIF、MP4。

`backend/src/storage.rs` 中，SVG 和 PDF 可直接生成；GIF 和 MP4 则通过 `ffmpeg` 生成，临时文件放在系统临时目录，结束后自动清理。

导出任务采用队列和 worker 并发控制，避免用户一次性导出过多任务拖垮服务。

6.7 管理后台与指标

管理员页面由 `backend/src/admin.rs` 和 `src/components/AdminConsole.jsx` 共同完成。后台可以看到：

1. 服务运行时间。
2. 模型配置状态。
3. 任务队列状态。
4. 缓存统计。
5. 接口耗时分布。
6. MongoDB 查询耗时分布。
7. 最近任务、最近资产和最近分享。

`backend/src/metrics.rs` 还统计了 P50、P95、P99 等指标，这比只看平均值更真实。

7. 系统运行流程

项目从打开页面到看到图形，大致经历以下步骤：

1. 浏览器加载 `index.html`，挂载 React 应用。
2. App 读取当前路由，决定显示概览、工作台、认证页还是后端页。
3. 用户在 Monaco 编辑器里输入 GeoGebra 脚本。
4. `Preprocessor` 清洗并验证脚本。
5. `Dispatcher` 按行执行命令并输出日志。
6. `GeoGebraEngine` 把命令发送给 GeoGebra 画布。

- 7. 用户拖动自由点时，系统把新的坐标识别为“脏状态”。
- 8. 用户可以同步回代码、保存项目、创建版本或导出结果。
- 9. 如果需要 AI 生成或团队协作，则通过后端 API 完成。

8. 遇到的问题与解决方案

问题	原因	解决方案
GeoGebra 加载慢或失败	<code>deployggb.js</code> 是异步加载，网络或 CDN 不稳定	用 <code>readyPromise</code> + <code>ggbOnInit</code> 等待就绪，并设置超时和重试提示
中文注释或中文文件名显示乱码	PowerShell 默认输出编码不是 UTF-8	在读取源码前固定 UTF-8 编码，见附录命令
指令在不同语言环境下失效	GeoGebra 有本地化命令	初始化时强制 <code>language: 'en'</code> ，统一使用英文 API
注释中的 <code>//</code> 被误删	简单字符串切割会误判字符串内部内容	先找 <code>//</code> ，再统计引号数量，判断是否在字符串中
<code>f(x)=...</code> 这类函数定义被误判	直接用 <code>=</code> 切分会把括号内部内容算错	通过深度扫描，只识别顶层赋值
旧图形残留影响新脚本	画布不是每次都干净	运行前先 <code>reset()</code> ，必要时 <code>clear()</code>
大脚本导致界面卡顿	单次执行量过大	预处理限长、调度器限时、日志分步输出
拖拽后代码不同步	只改画布，没有回写脚本	用 <code>movedGeos</code> 监听拖拽，并导出自由点命令回写
外部模型或接口失败	网络、密钥、返回格式都可能出错	模型失败时回退到本地 fallback，导出和上传也做错误包裹
GIF/MP4 导出失败	依赖 <code>ffmpeg</code> 或临时目录	把导出放进独立任务，失败时返回明确错误并清理临时文件
登录态过期	会话 token 有生命周期	统一在 API 层处理 401，并清理本地会话

PowerShell 乱码排查说明

你提到的“查看代码乱码”通常不是文件损坏，而是 PowerShell 输出编码问题。建议在查看 UTF-8 源码前先执行：

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
$OutputEncoding = [System.Text.Encoding]::UTF8
Get-Content -Encoding UTF8 src\components\App.jsx
```

如果你要看别的文件，把路径替换成实际文件即可。本项目中建议统一按 UTF-8 读取前端和后端源码。

9. 测试与验证

本项目的验证思路不是只看“能不能打开”，而是分层检查：

- 1. 前端用 Vite 开发服务器验证热更新和页面渲染。
- 2. Monaco 编辑器验证语法高亮、默认模板和只读切换。

3. GeoGebra 画布验证异步加载、执行、重置和导出。
4. 后端验证登录、项目、AI、导出和管理员接口。
5. MongoDB 配置时验证索引、会话和任务持久化。

如果作为课程项目提交，建议再补充两类测试：

1. 典型脚本测试，例如点、线、圆、三角形和中点。
2. 异常脚本测试，例如空行、注释、括号错误和未定义变量。

10. 部署说明

开发环境下，前端通过 Vite 启动，默认端口是 5173。如果没有设置 `VITE_API_BASE_URL`，开发服务器会把 `/api`、`/health` 和 `/assets` 转发到后端。

生产环境下：

1. 前端先执行 `npm run build`。
2. 后端可通过 `FRONTEND_DIST_DIR` 读取 `dist` 并提供静态资源。
3. `Dockerfile` 和 `docker-compose.yml` 可用于容器化部署。
4. `index.html` 的 CSP 已允许 GeoGebra 所需脚本和 frame 域名。

11. 设计收获与反思

这个项目最大的收获不是“做出了一个绘图工具”，而是学会了如何把一个复杂系统拆成多个层次：

1. 输入层：编辑器和表单。
2. 解释层：预处理器和日志。
3. 执行层：GeoGebra 引擎。
4. 服务层：Rust 后端和 AI。
5. 存储层：本地存储、MongoDB 和临时导出文件。

同时也认识到，真正稳定的软件不仅要“能用”，还要“出错时说得清楚、恢复得回来、日志查得到、数据存得住”。

12. 后续扩展方向

如果继续迭代，这个项目还可以扩展为：

1. 更强的图形识别，把手绘图直接转成脚本。
2. 更完整的版本 diff 和回滚。
3. 更细的团队权限和审阅流程。
4. 更多导出格式，例如 PPTX 和 GeoGebra 原生文件。
5. 更智能的教学讲解，自动生成课堂讲稿。
6. 更多课堂模板，方便初中几何教学直接使用。

13. 代码依据对应表

下面这些文件是本报告的主要依据：

文件	作用
<code>src/components/App.jsx</code>	页面总入口、全局状态、业务编排

文件	作用
src/components/CodeEditor.jsx	编辑器壳层和按需加载
src/components/MonacoCodeEditor.jsx	Monaco 语言、主题和 Worker 配置
src/components/GeoGebraContainer.jsx	GeoGebra 画布加载与骨架屏
src/components/ControlPanel.jsx	运行、清空、导出、重置与模式切换
src/components/LogPanel.jsx	执行日志和错误展示
src/engine/GeoGebraEngine.js	GeoGebra API 封装、执行、样式、拖拽监听
src/engine/Preprocessor.js	注释清洗、语法校验、安全检查
src/engine/Dispatcher.js	顺序执行和报告生成
src/api/backend.js	前端请求封装、认证、轮询、上传和导出
src/utils/studio.js	项目、版本、参数和拖拽同步工具
src/utils/auth.js	本地会话存储
src/utils/appRoutes.js	页面路由与分享入口
backend/src/app.rs	后端启动与监听
backend/src/config.rs	环境变量配置
backend/src/http/handlers.rs	HTTP 路由和业务接口
backend/src/types.rs	请求、响应和数据模型
backend/src/auth.rs	注册、登录、权限校验
backend/src/store/mod.rs	内存/数据库读写桥接
backend/src/store/mongo.rs	MongoDB 持久化与索引
backend/src/model.rs	AI 模型调用
backend/src/storage.rs	SVG/PDF/媒体导出
backend/src/admin.rs	管理后台快照
backend/src/metrics.rs	指标统计
index.html	CSP、GeoGebra 脚本和页面基础结构
vite.config.js	开发代理、构建配置与基路径

结语

本项目把几何作图、脚本编写、日志分析、版本管理、AI 辅助和后台运维合并到同一个工作台中，形成了一个“能教、能画、能管、能扩展”的综合科技编程系统。对于初中综合科技项目来说，它既有数学内容，也有编程思维，还包含工程化、数据安全和系统设计，适合作为实践报告和成果展示文档。